رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

DEPI Graduation Project

Predicting Customer Churn

in Telecommunications

May 16, 2025.

Our Team

- Noran Ali ElSaeid
- Donia Wael Mohamed
- Aya Elsayed Ali
- Ibrahim Mohamed
- Ahmed Abdullah

Agenda

BUSINESS PITCH

1. Introduction
2. Problem Statement
3. Data Collection
4. Exploratory Data Analysis
5. Preprocessing & Feature Engineeering
6. Dashboard & Advanced Visualization
7. Modeling Approaches
8. Model Deployment

Introduction

- **Customer churn is a major concern in the competitive telecommunications industry, as it directly affects revenue and long-term profitability. This project analyzes real-world data from a telecom provider to understand the key factors driving customer attrition. Using exploratory data analysis (EDA), feature engineering, and machine learning, the goal is to identify customers at risk of churning and offer actionable strategies to improve customer retention and satisfaction.**

Problem Statement

This project aims to solve two main problems for telecom companies:

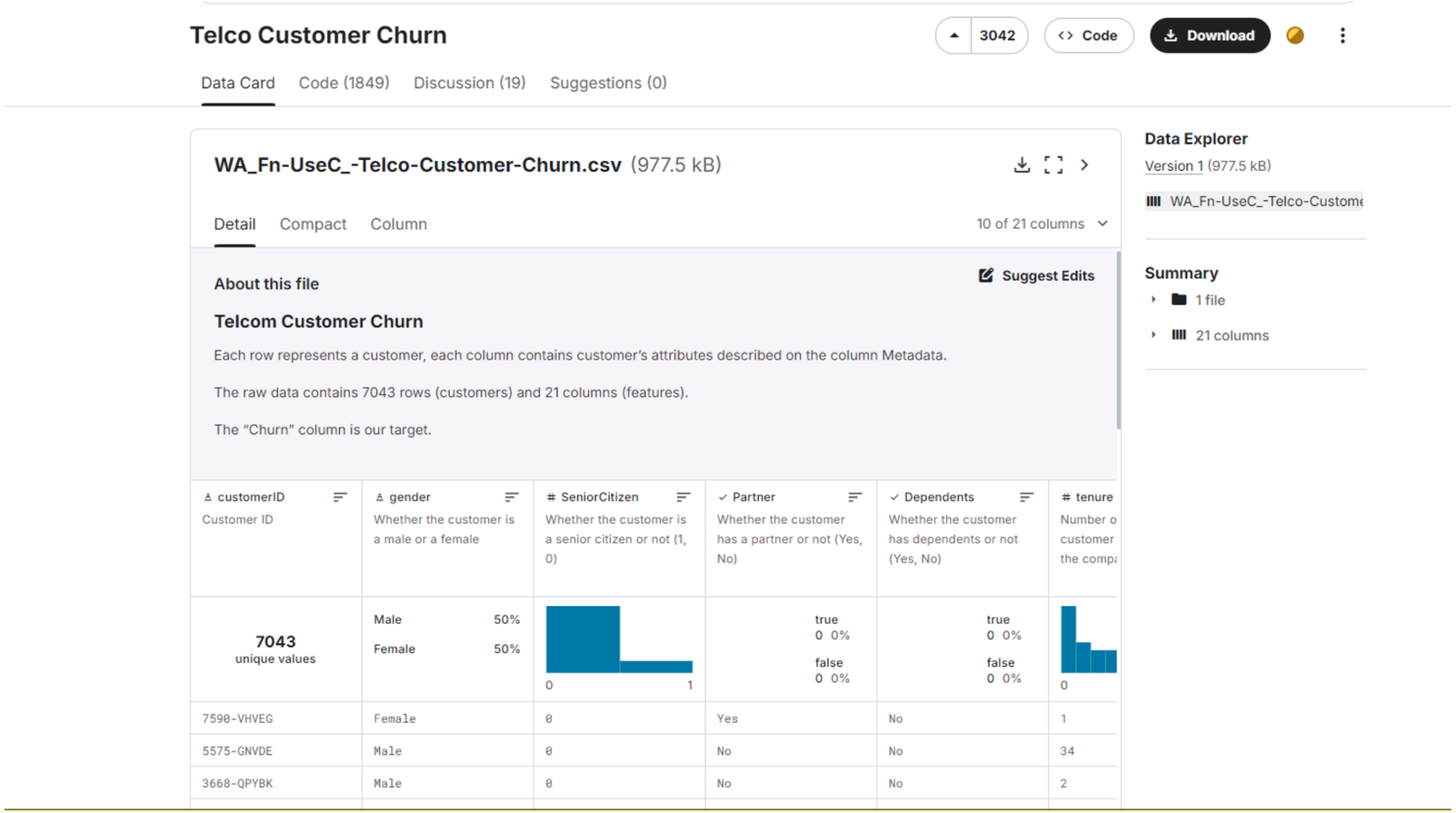
- **Which customers are likely to leave?**
- **Why do they leave?**

Understanding these helps the company to:

- **Detect early signs of churn.**
- **Offer special deals to retain customers.**
- **Improve services based on customer needs.**

Data Collection

- **The dataset used in this study is the Telco Customer Churn Dataset , sourced from Kaggle and originally provided by IBM Watson Analytics Lab.**



Exploratory Data Analysis

Data Preprocessing

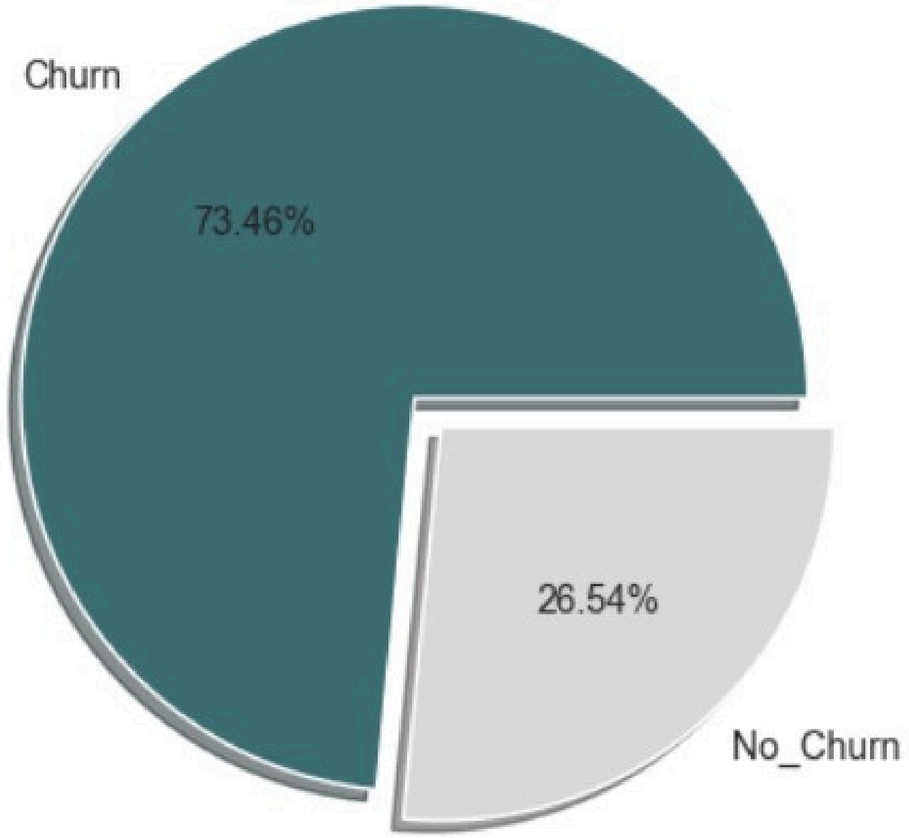
- Drop the customerID column because not important
- Handle blank spaces in TotalCharges:
  - Replace blanks with NaN
  - Drop missing rows
- Convert values in certain columns from categorical to integer
- Apply Label Encoding to categorical columns
- Apply min-max scaling to the columns: 'tenure,' 'MonthlyCharges,' and 'TotalCharge'
- Split the data into "X" and "Y"
- Split 'X','Y' into x_train , y_train,x_test,y_test

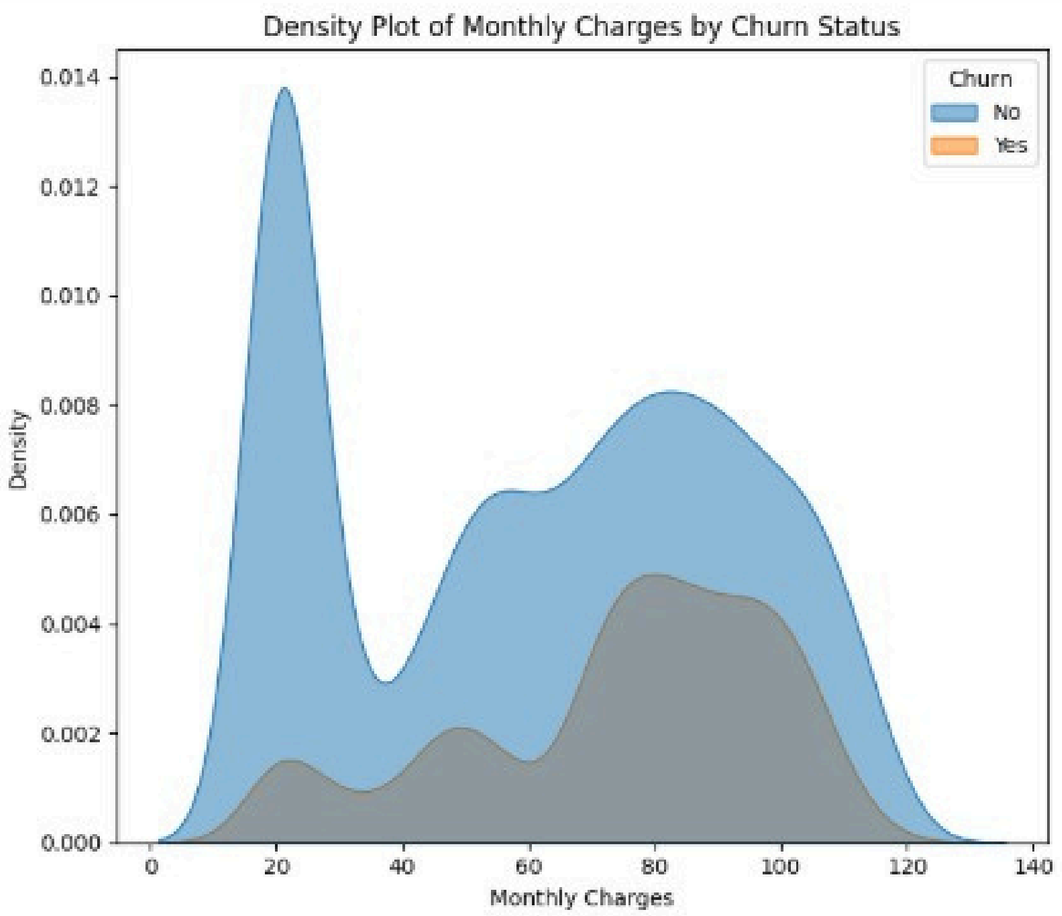
DATA EXPLORATION

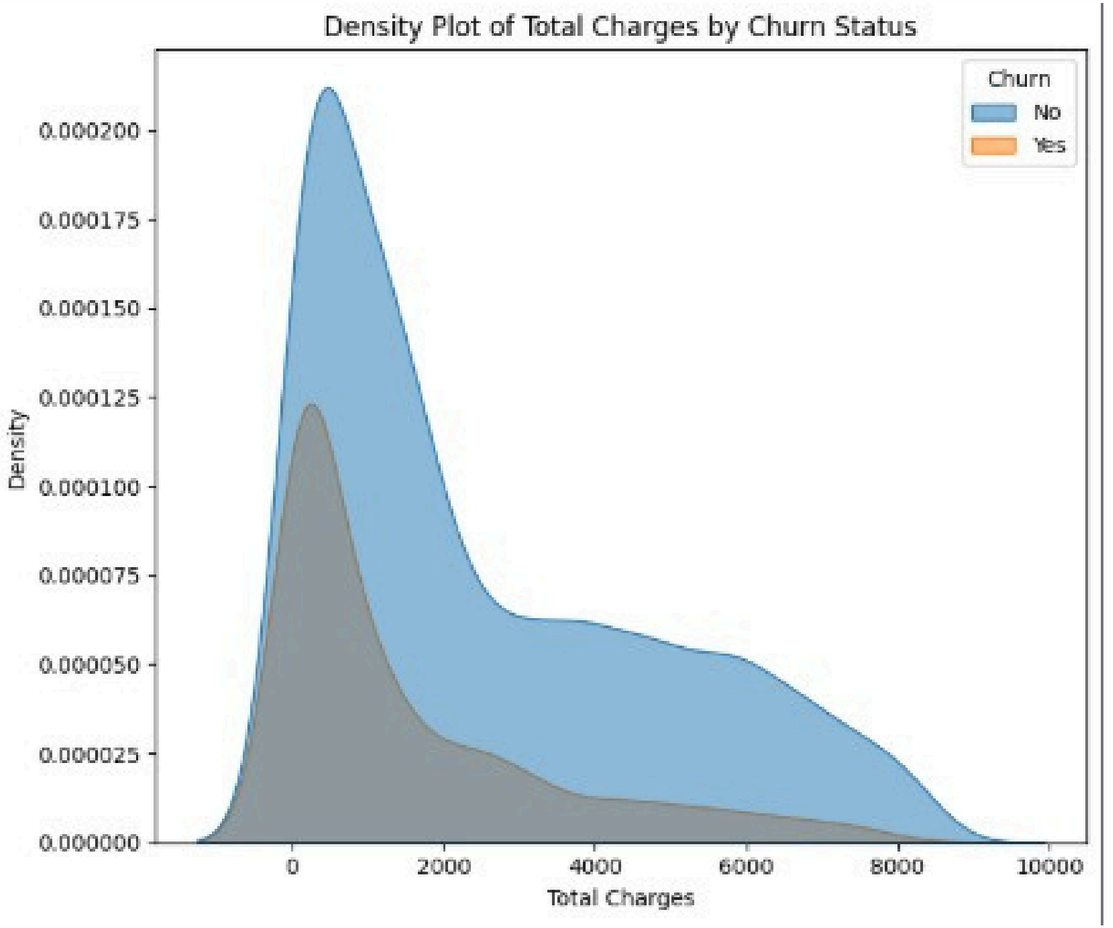
About 27% of customers have churned

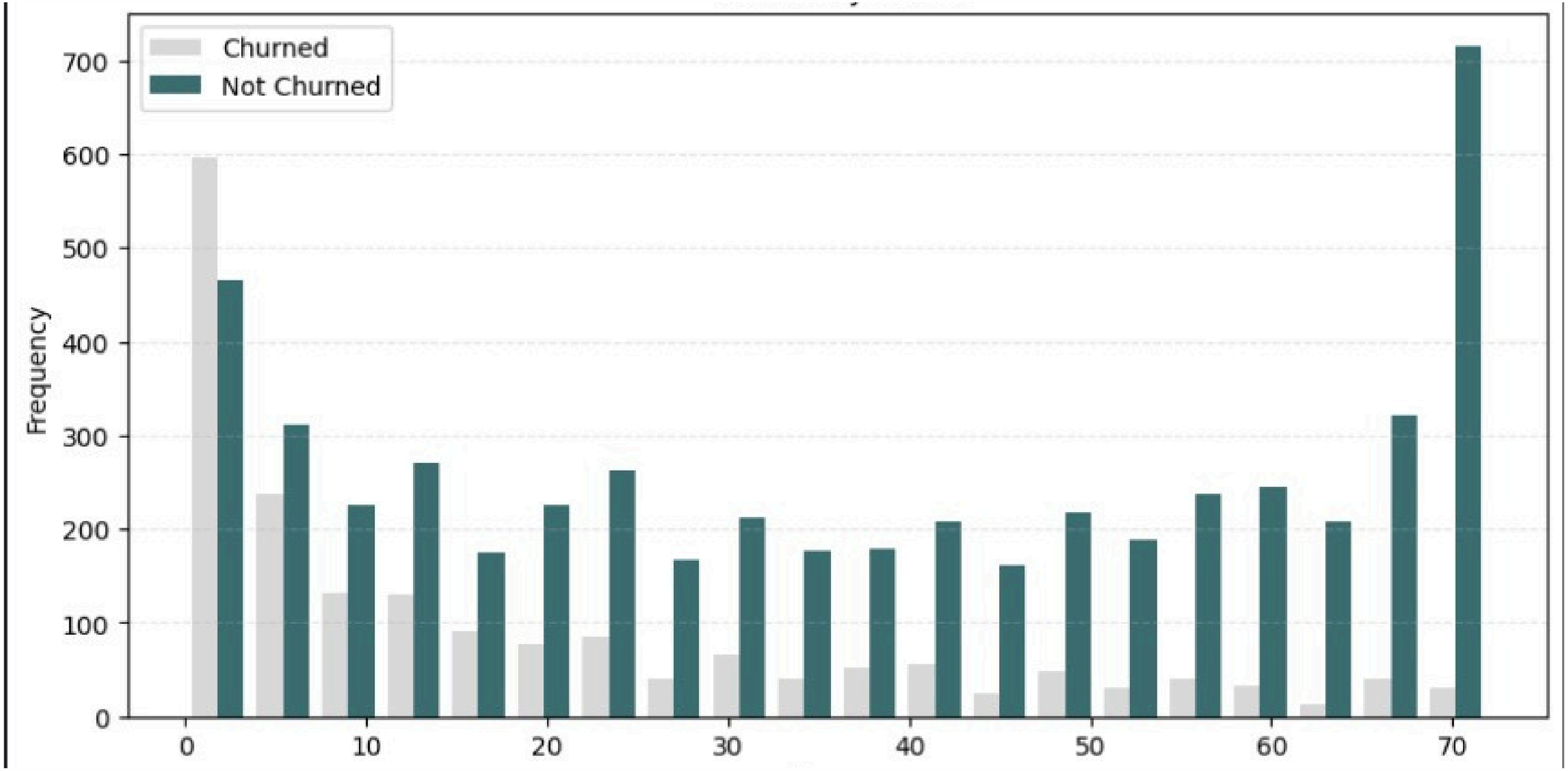
Indicates class imbalance

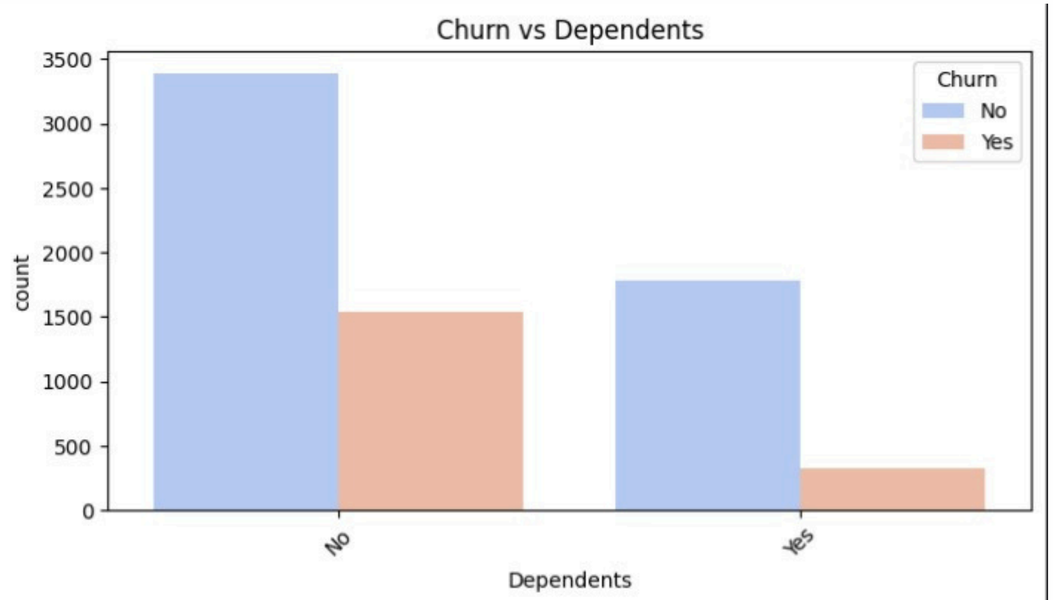
will apply techniques like SMOTE later

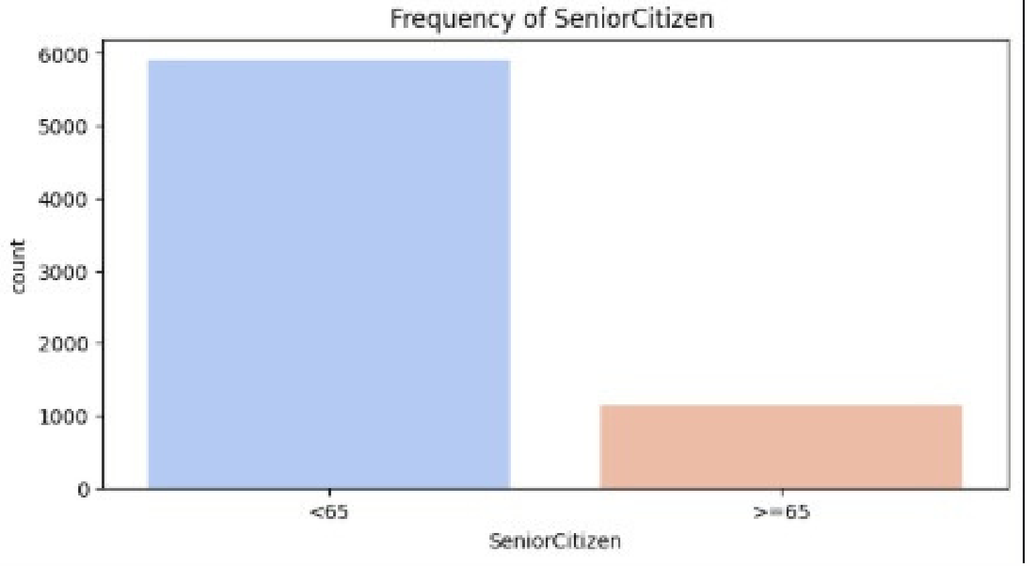


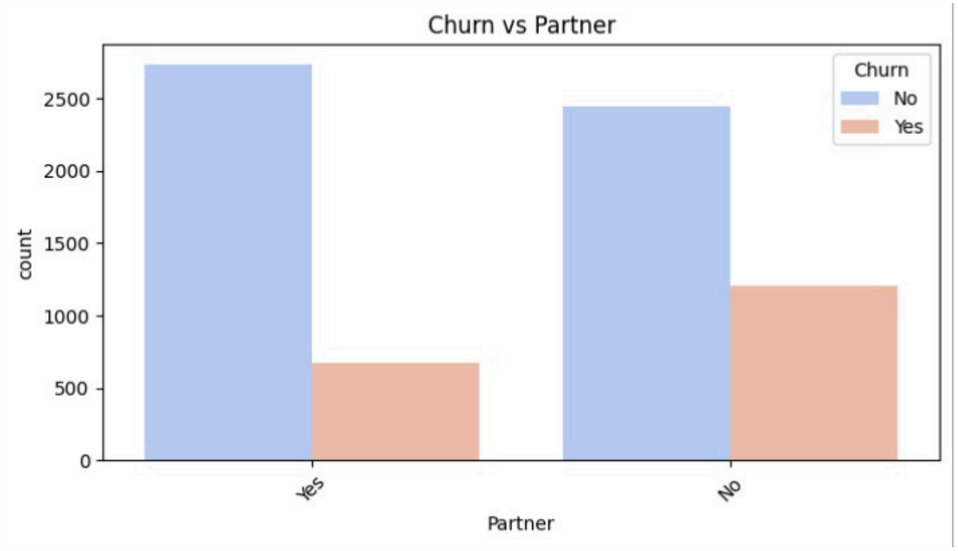

Density Plot of Monthly Charges by Churn Status
Churn
No
Yes
Density
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
0
20
40
60
80
100
120
140
Monthly Charges

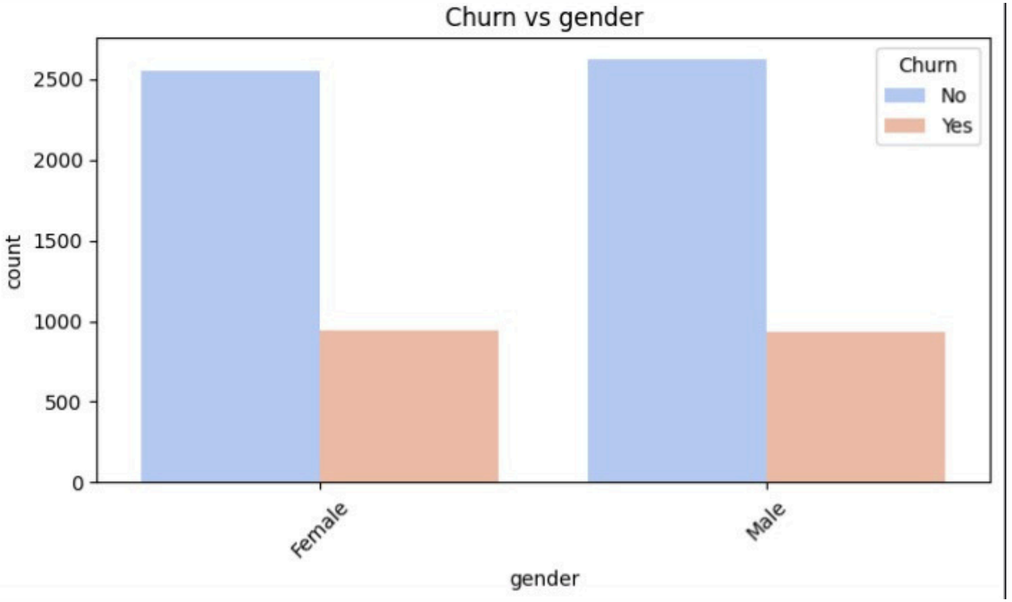

Density Plot of Total Charges by Churn Status
Churn
No
Yes
Density
0.000200
0.000175
0.000150
0.000125
0.000100
0.000075
0.000050
0.000025
0.000000
0
2000
4000
6000
8000
10000
Total Charges

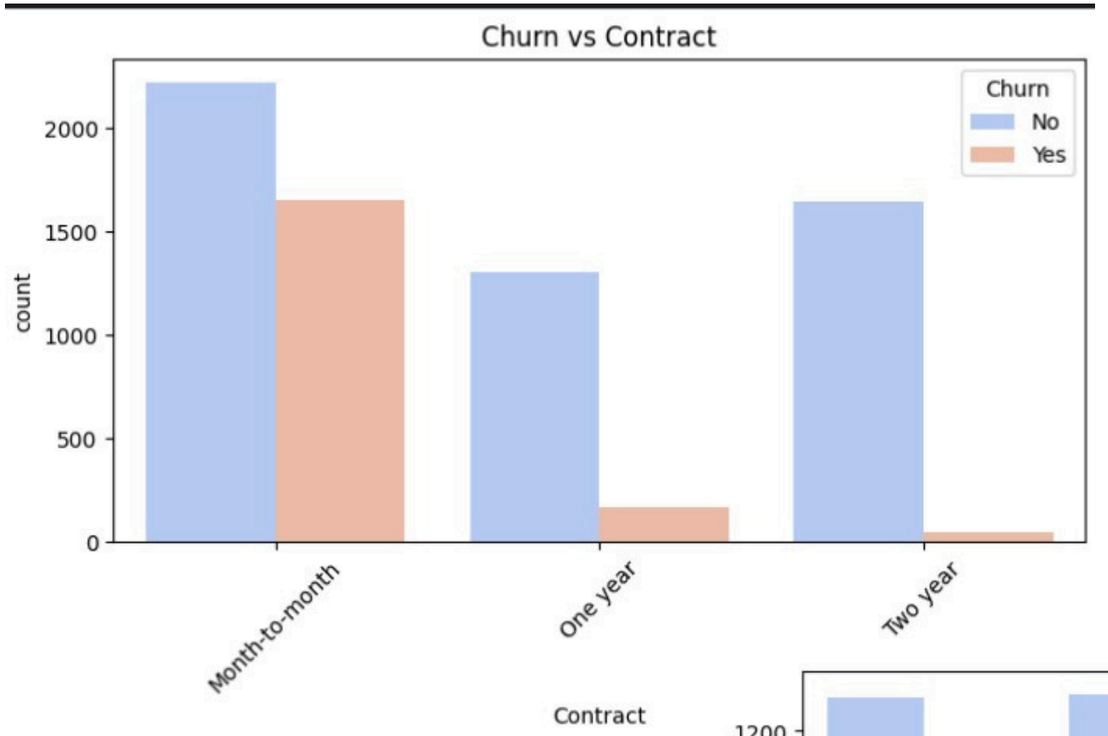

Churned
Not Churned
Frequency
0
100
200
300
400
500
600
700
0
10
20
30
40
50
60
70

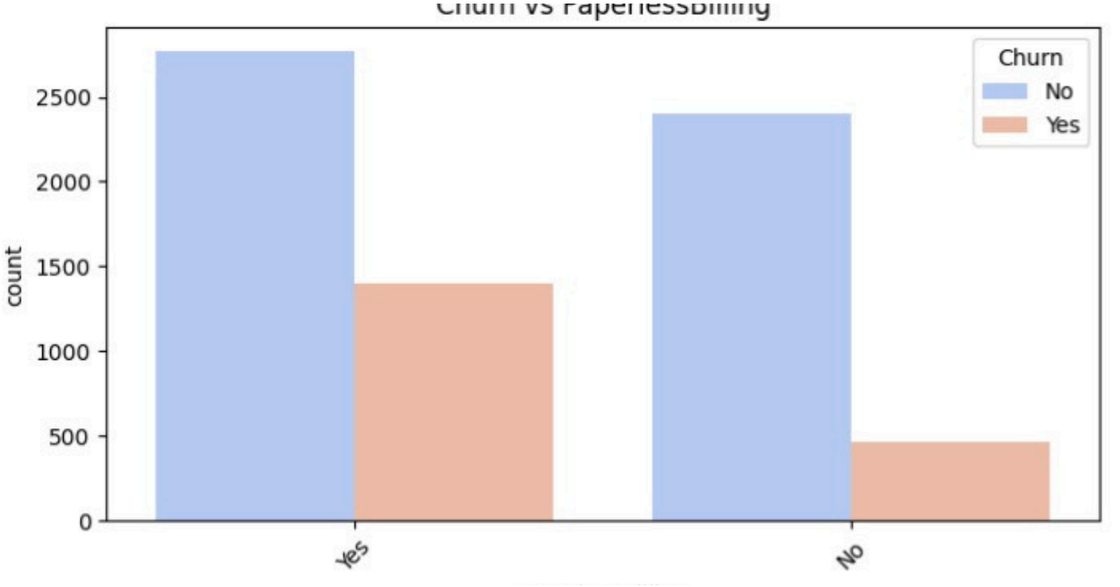

Churn vs Dependents
Churn
No
Yes
count
Dependents
No
Yes
0
500
1000
1500
2000
2500
3000
3500

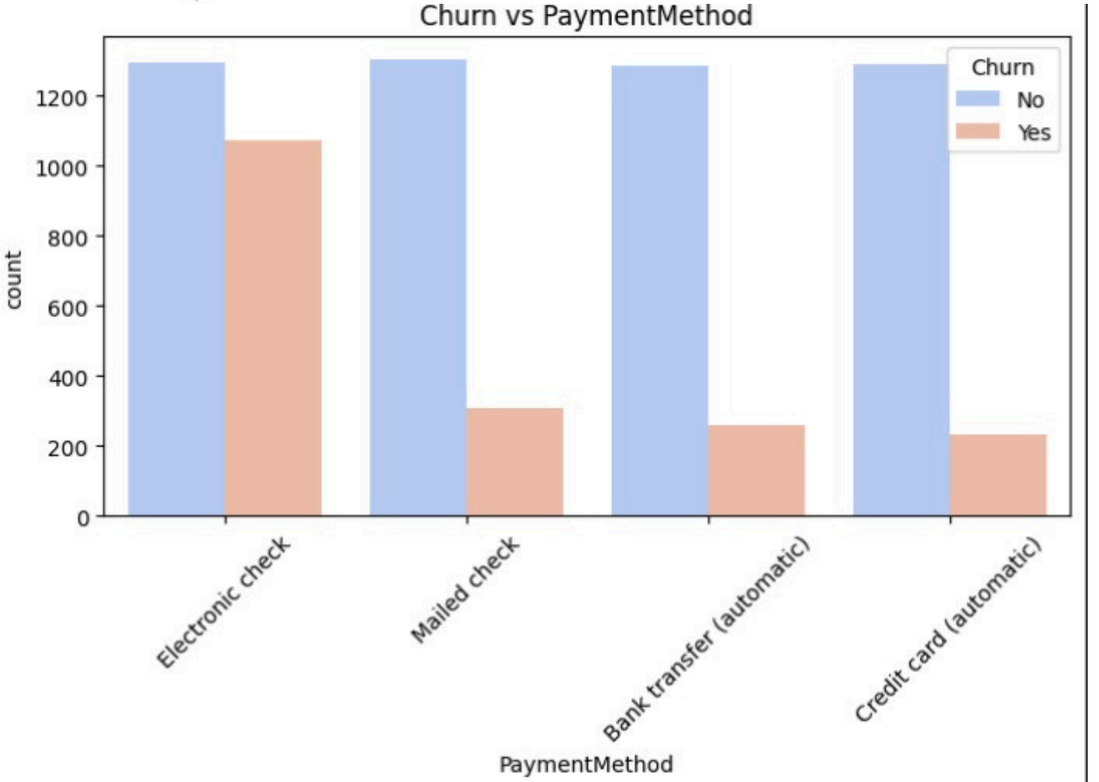

Frequency of SeniorCitizen
count
SeniorCitizen
<65
>=65
0
1000
2000
3000
4000
5000
6000

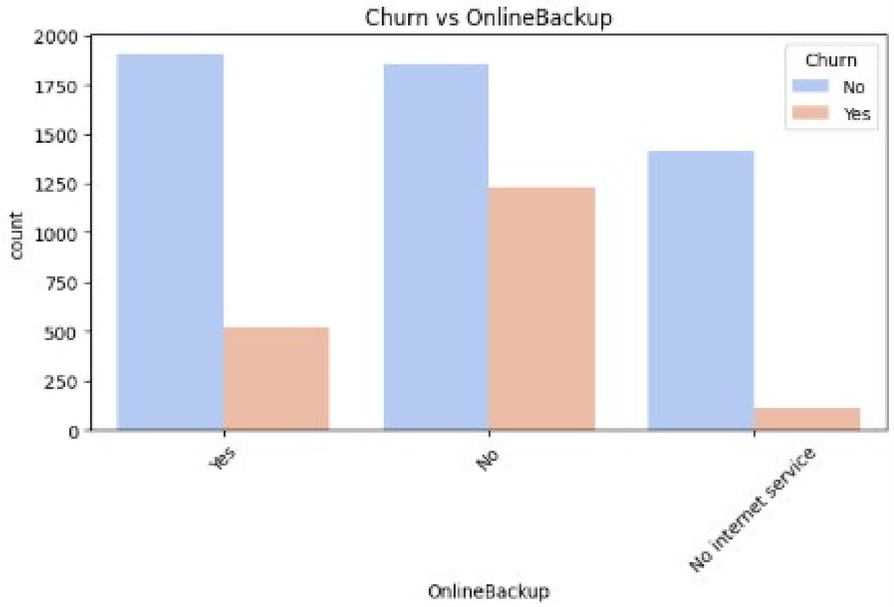

Churn vs Partner
Churn
No
Yes
count
Partner
Yes
No
0
500
1000
1500
2000
2500

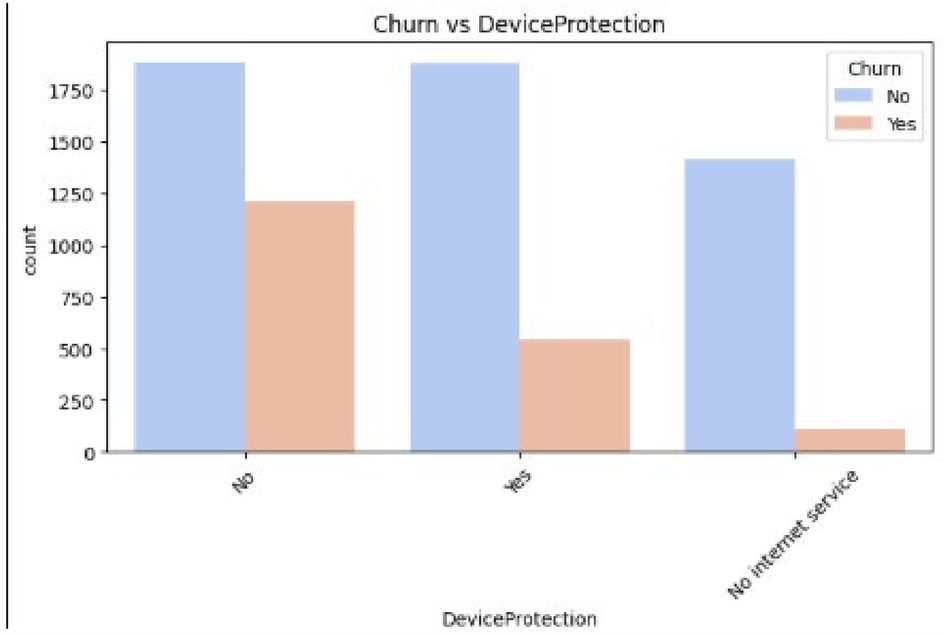

Churn vs gender
Churn
No
Yes
count
gender
Female
Male
0
500
1000
1500
2000
2500

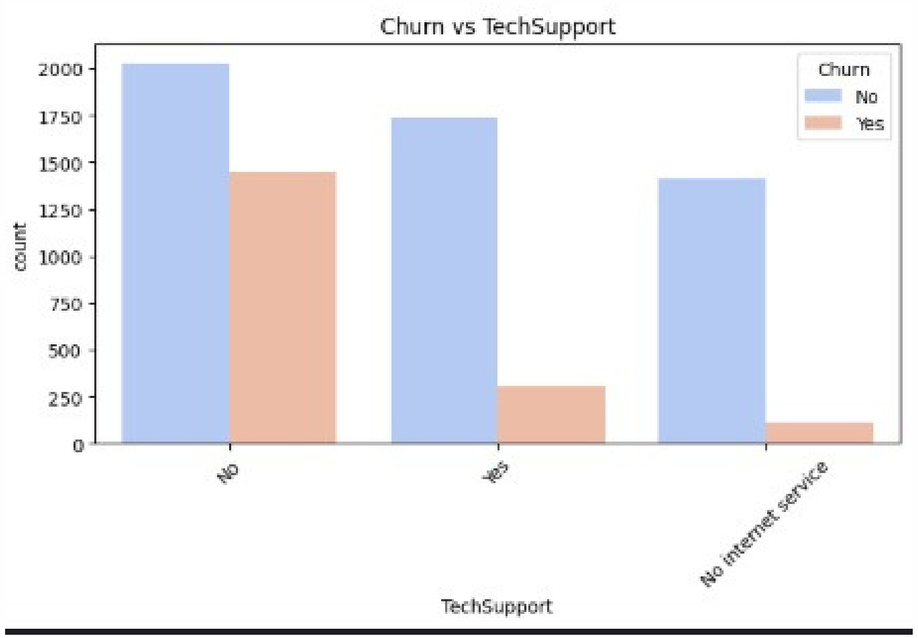

Churn vs Contract
Churn
No
Yes
count
2000
1500
1000
500
0
Month-to-month
One year
Two year
Contract

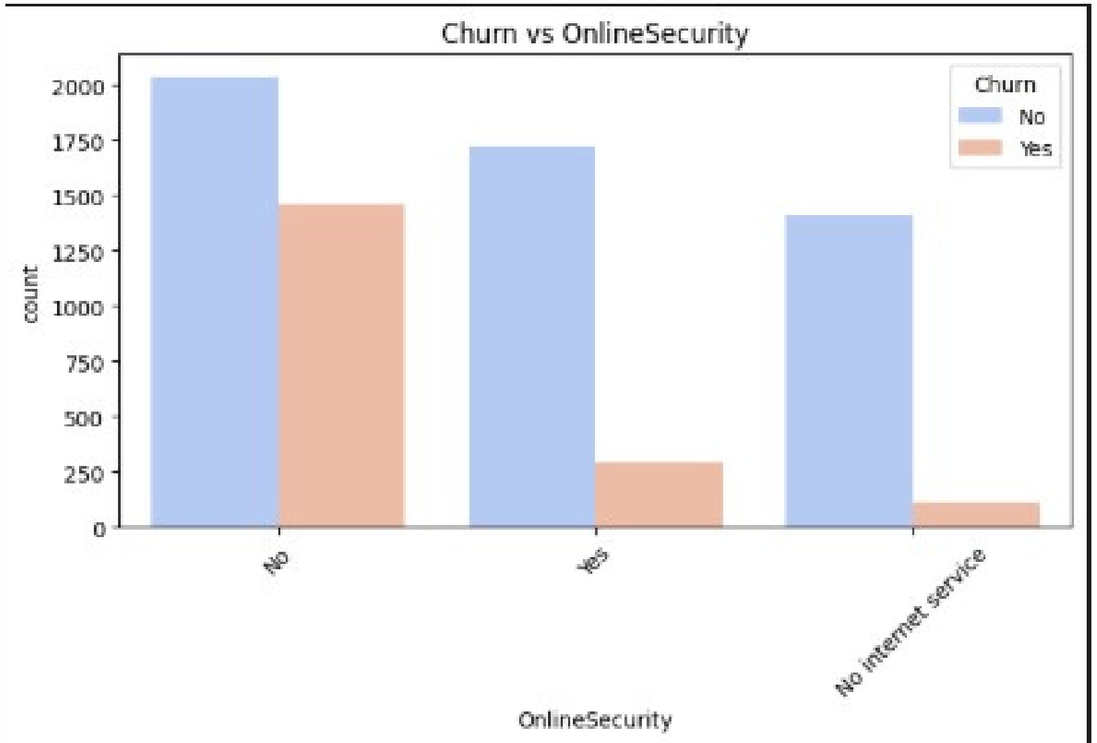

Churn
No
Yes
count
2500
2000
1500
1000
500
0
Yes
No


Churn vs PaymentMethod
Churn
No
Yes
count
1200
1000
800
600
400
200
0
Electronic check
Mailed check
Bank transfer (automatic)
Credit card (automatic)
PaymentMethod


Churn vs OnlineBackup
Churn
No
Yes
count
2000
1750
1500
1250
1000
750
500
250
0
Yes
No
No internet service
OnlineBackup


Churn vs DeviceProtection
Churn
No
Yes
count
1750
1500
1250
1000
750
500
250
0
No
Yes
No internet service
DeviceProtection


Churn vs TechSupport
Churn
No
Yes
count
2000
1750
1500
1250
1000
750
500
250
0
No
Yes
No internet service
TechSupport


Churn vs OnlineSecurity
Churn
No
Yes
count
2000
1750
1500
1250
1000
750
500
250
0
No
Yes
No internet service
OnlineSecurity

Feature Engineering

Feature Engineering Process

- Created meaningful flags:

  has_protection, has_streaming
- Created new metrics:

  TotalPaid, spend_trend, service_adoption_rate
- Total services used per customer

Feature Engineering Process

Create new features

```
[40] # Create security services flag
     df['has_protection'] = ((df['OnlineSecurity'] == 'Yes') |
                             (df['OnlineBackup'] == 'Yes') |
                             (df['DeviceProtection'] == 'Yes') |
                             (df['TechSupport'] == 'Yes')).astype(int)
```

```
[41] # Create streaming services flag
     df['has_streaming'] = ((df['StreamingTV'] == 'Yes') |
                            (df['StreamingMovies'] == 'Yes')).astype(int)
```

```
[42] df['TotalPaid'] = df['MonthlyCharges'] * df['tenure']
```

```
     # Calculate spend trend (comparison of current monthly charges to average monthly spend)
     df['avg_monthly_spend'] = df['TotalCharges'] / (df['tenure'] + 0.01)
     df['spend_trend'] = df['MonthlyCharges'] / (df['avg_monthly_spend'] + 0.01)
     # This feature measures whether the customer's spending is increasing or decreasing over time.
     # A higher value indicates that the customer's spending is increasing.
```

```
[44] # Count total number of services the customer uses
     service_columns = ['PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
                        'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']

     df['total_services'] = df[service_columns].sum(axis=1)

     # Calculate service adoption rate (services per tenure)
     df['service_adoption_rate'] = df['total_services'] / (df['tenure'] + 1)
     # This feature measures how many services the customer has adopted relative to their tenure.
     # It helps to assess the level of engagement with available services.
```

Feature Selection Techniques

Feature Selection Techniques

- Applied 4 advanced techniques:
  - Chi-Square,
  - ANOVA (F-test),
  - RFE(select best feature based on ML model),
  - Mutual Info
- Selected top 10 features from each
- Kept features appearing in ≥ 3 methods

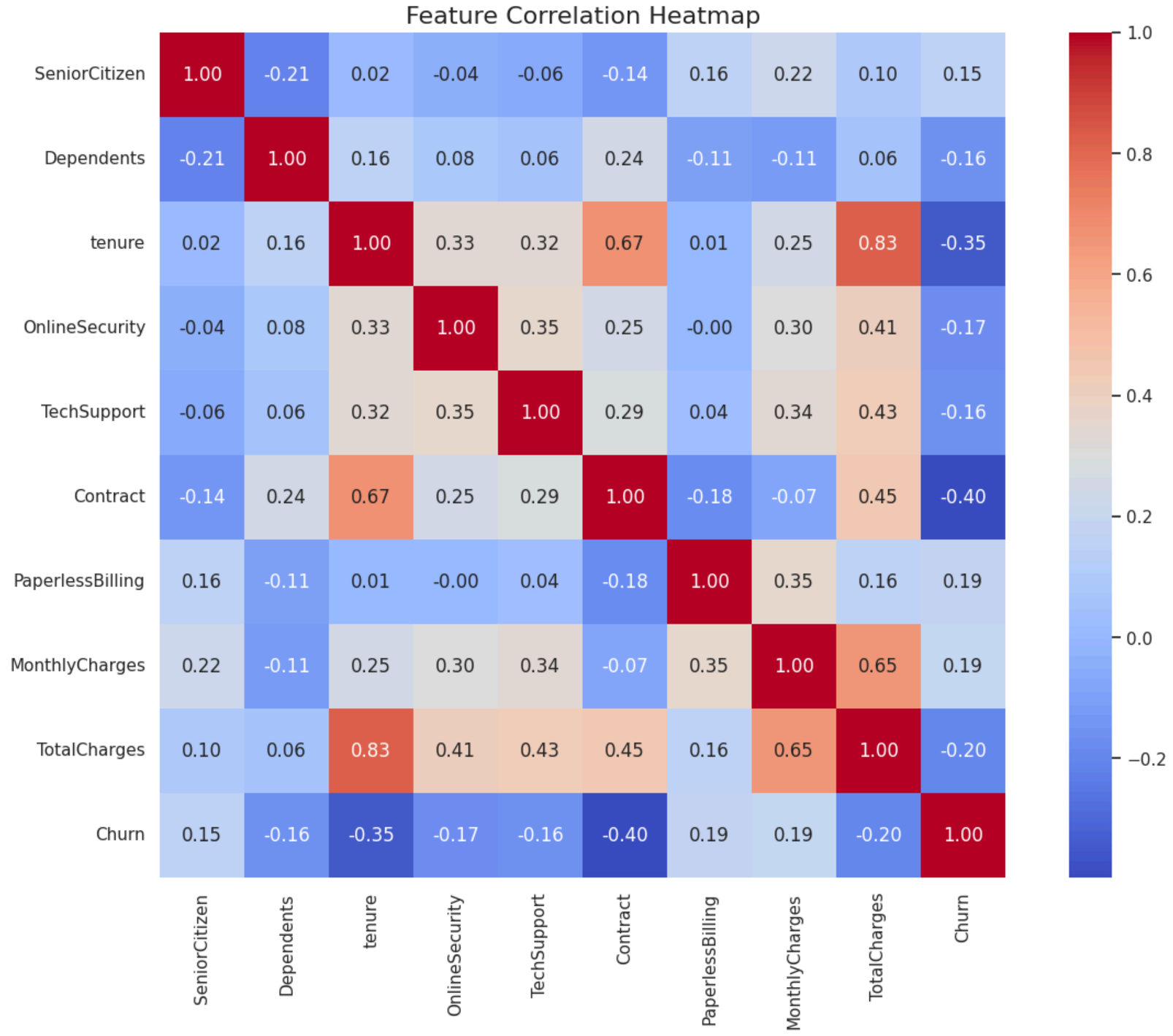
Common Features

Features that appeared in at least 3 out of 4 methods

```
# Features selected by Chi-Square test
features_chi2 = selected_features.tolist()

# Features selected by ANOVA with F-Score high and P-Value < 0.05
features_anova = anova_scores[anova_scores['P-Value'] < 0.05]['Feature'].tolist()

# Features selected by RFE (features with ranking = 1)
features_rfe = rfe_results[rfe_results['Selected'] == True]['Feature'].tolist()

# Features selected by Mutual Information (top 10)
features_mi = mi_results.head(10)['Feature'].tolist()
```

```
from collections import Counter

# Combine all features into one list
all_features = features_chi2 + features_anova + features_rfe + features_mi

# Count how many times each feature appears
feature_counts = Counter(all_features)

# Get features that appeared in at least 3 methods
features_in_3_or_more = [feature for feature, count in feature_counts.items() if count >= 3]

print("Features that appeared in at least 3 out of 4 methods:", features_in_3_or_more)
```

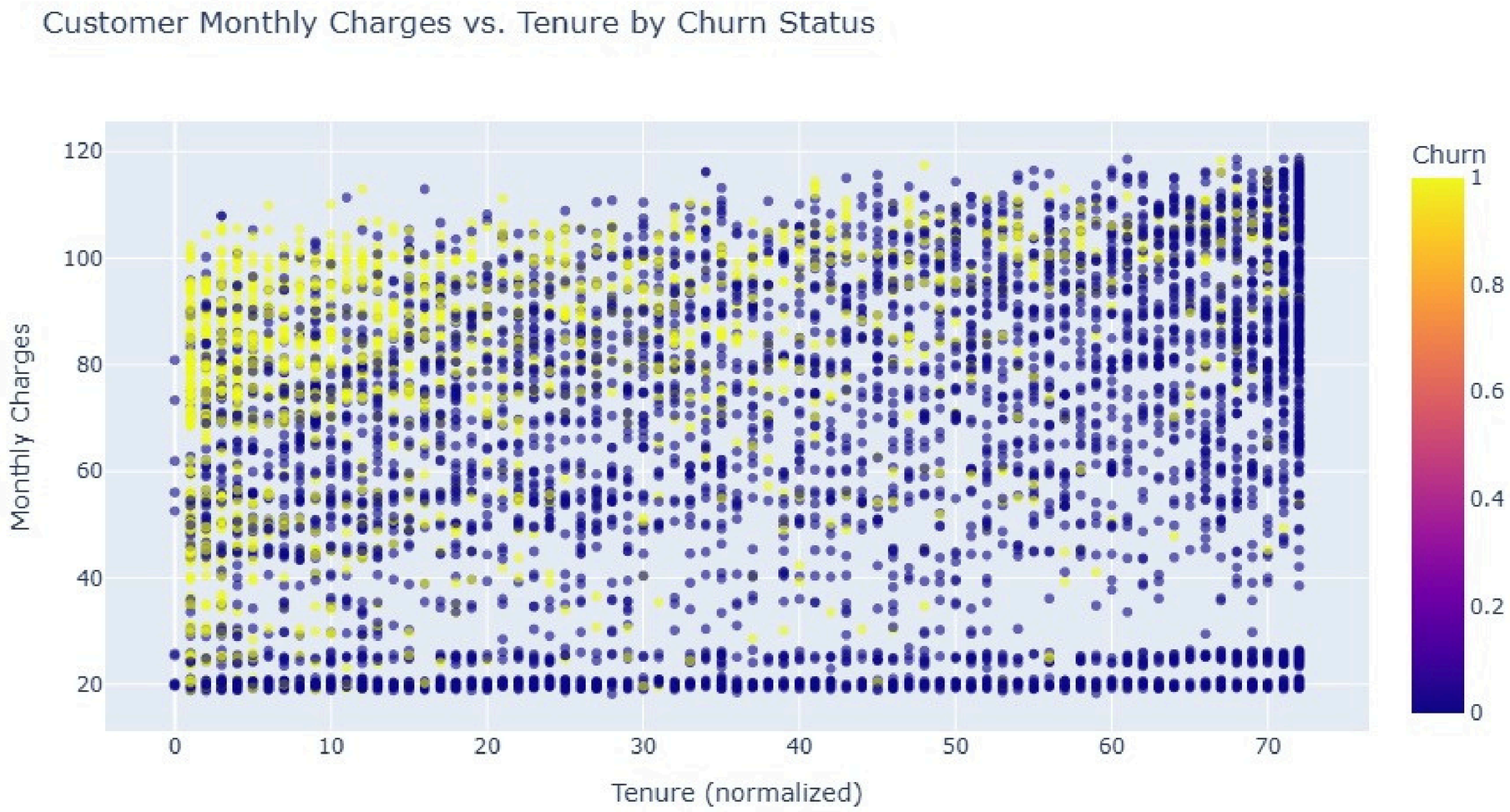
```
Features that appeared in at least 3 out of 4 methods: ['SeniorCitizen', 'tenure', 'OnlineSecurity', 'TechSupport', 'Contract', 'PaperlessBilling', 'TotalCharges', 'MonthlyCharges']
```

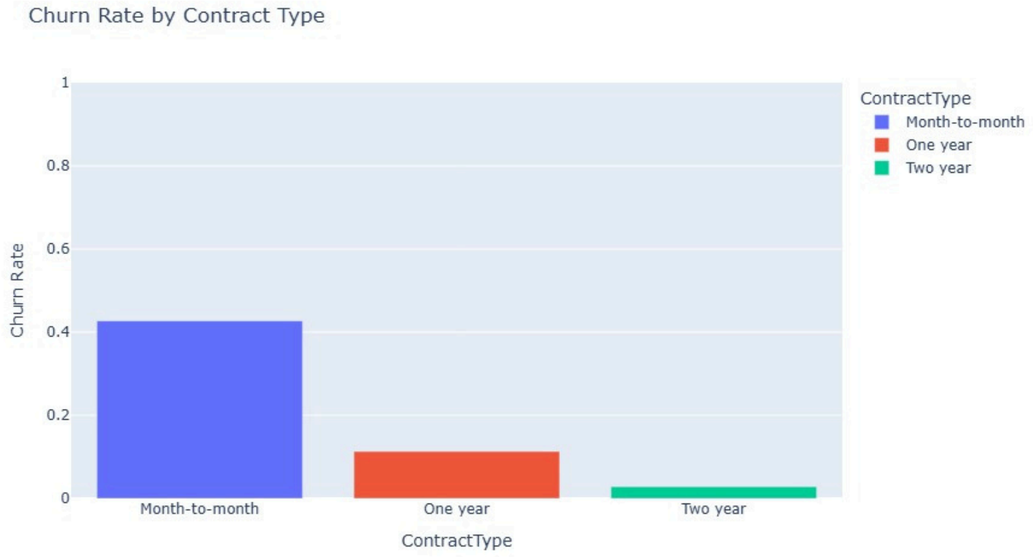

Feature Correlation Heatmap
SeniorCitizen 1.00 -0.21 0.02 -0.04 -0.06 -0.14 0.16 0.22 0.10 0.15
Dependents -0.21 1.00 0.16 0.08 0.06 0.24 -0.11 -0.11 0.06 -0.16
tenure 0.02 0.16 1.00 0.33 0.32 0.67 0.01 0.25 0.83 -0.35
OnlineSecurity -0.04 0.08 0.33 1.00 0.35 0.25 -0.00 0.30 0.41 -0.17
TechSupport -0.06 0.06 0.32 0.35 1.00 0.29 0.04 0.34 0.43 -0.16
Contract -0.14 0.24 0.67 0.25 0.29 1.00 -0.18 -0.07 0.45 -0.40
PaperlessBilling 0.16 -0.11 0.01 -0.00 0.04 -0.18 1.00 0.35 0.16 0.19
MonthlyCharges 0.22 -0.11 0.25 0.30 0.34 -0.07 0.35 1.00 0.65 0.19
TotalCharges 0.10 0.06 0.83 0.41 0.43 0.45 0.16 0.65 1.00 -0.20
Churn 0.15 -0.16 -0.35 -0.17 -0.16 -0.40 0.19 0.19 -0.20 1.00
SeniorCitizen
Dependents
tenure
OnlineSecurity
TechSupport
Contract
PaperlessBilling
MonthlyCharges
TotalCharges
Churn
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

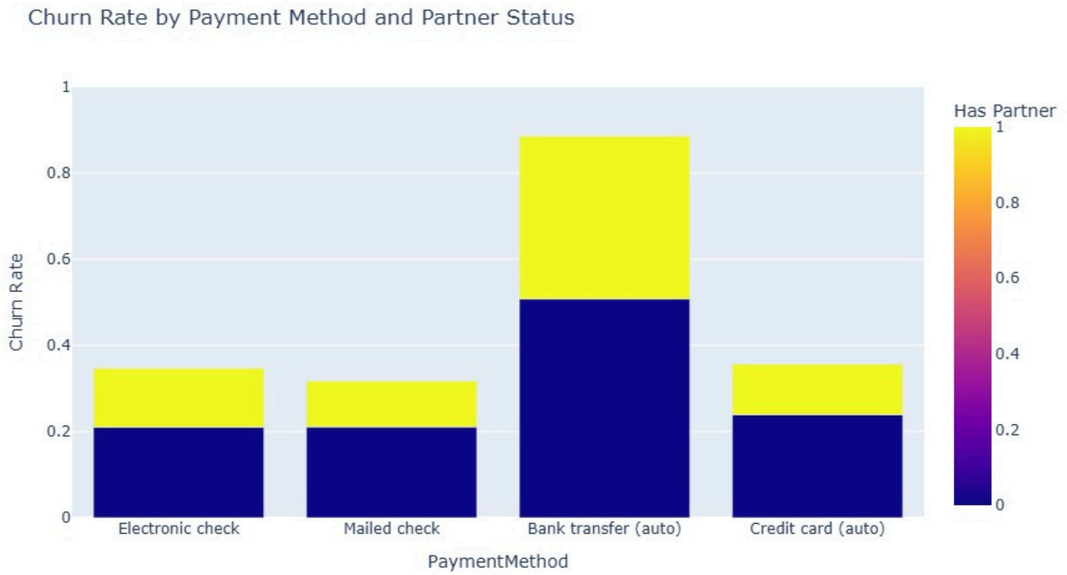
Outcome

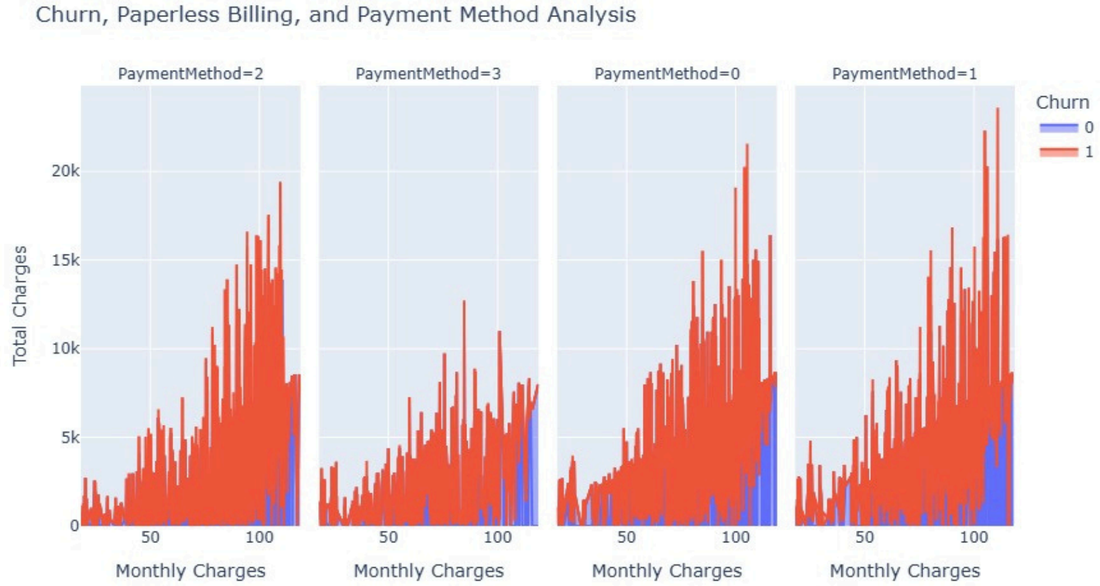
- Enhanced feature set improved model interpretability
- Selected features are meaningful and actionable
- Ready for final model training and dashboard deployment

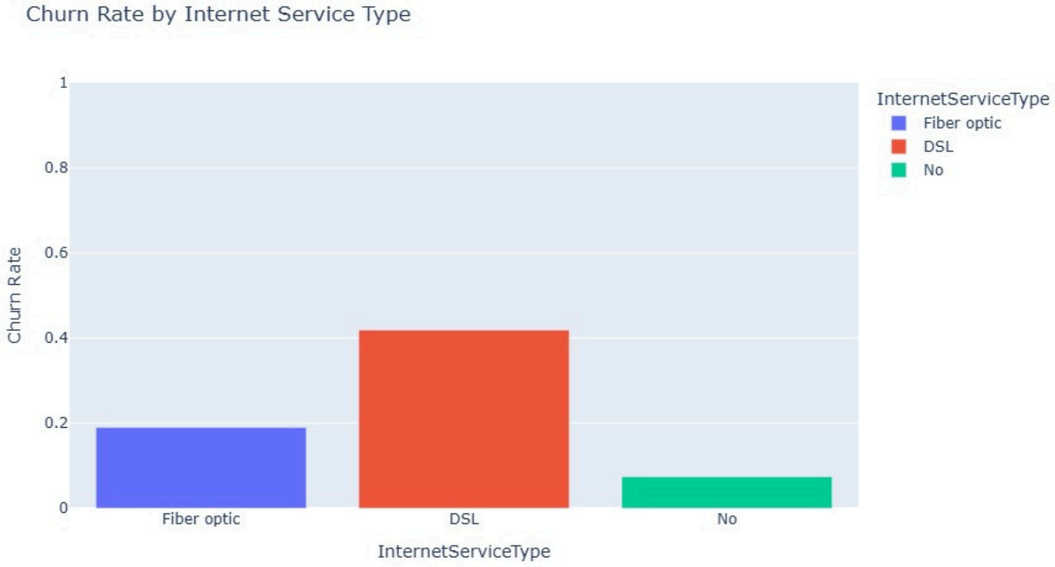
Advanced Visulaization

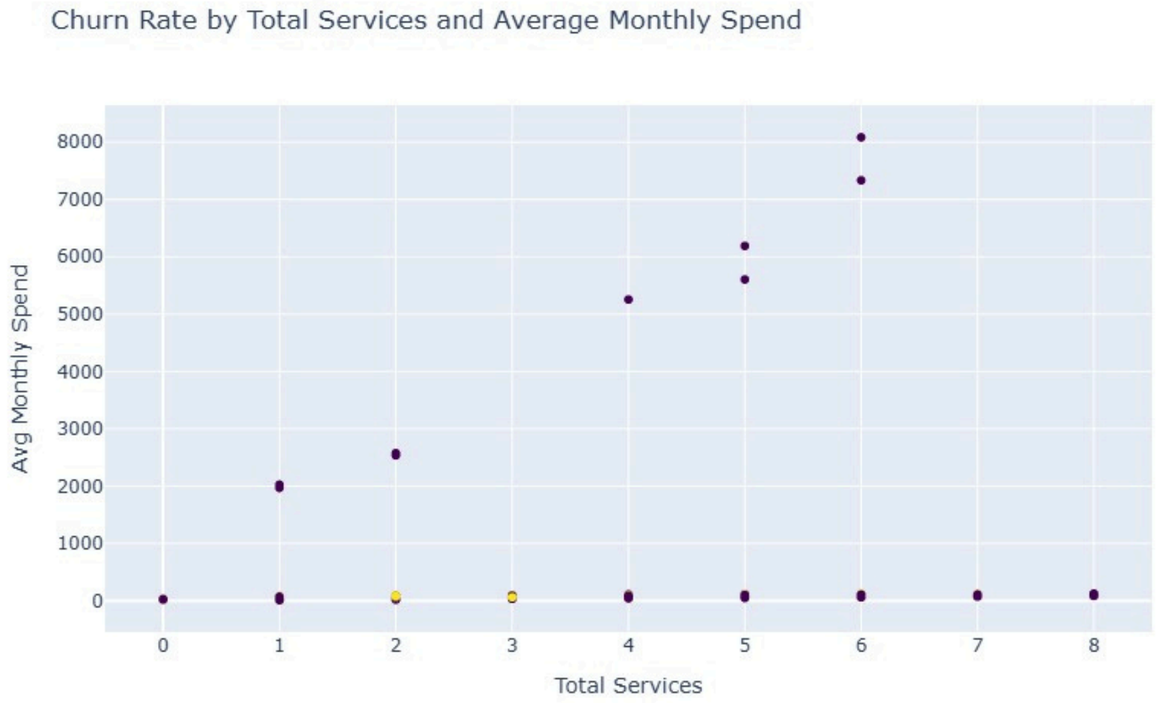


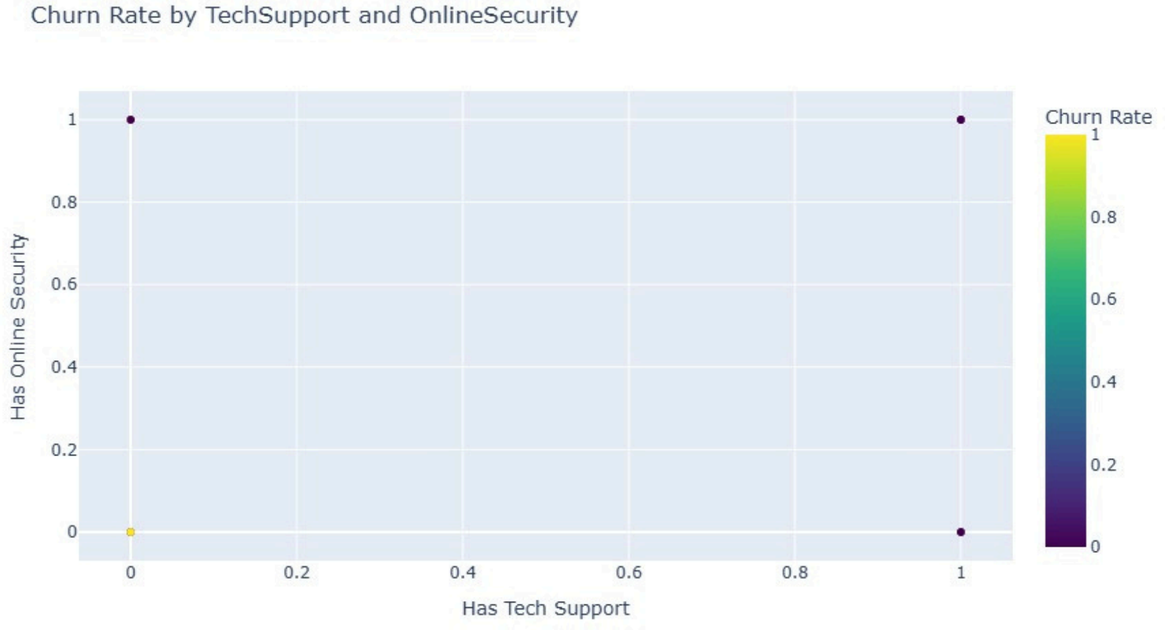

Churn Rate by Contract Type
Churn Rate
1
0.8
0.6
0.4
0.2
0
Month-to-month
One year
Two year
ContractType
ContractType
Month-to-month
One year
Two year


Churn Rate by Payment Method and Partner Status
Churn Rate
1
0.8
0.6
0.4
0.2
0
Electronic check
Mailed check
Bank transfer (auto)
Credit card (auto)
PaymentMethod
Has Partner
1
0.8
0.6
0.4
0.2
0

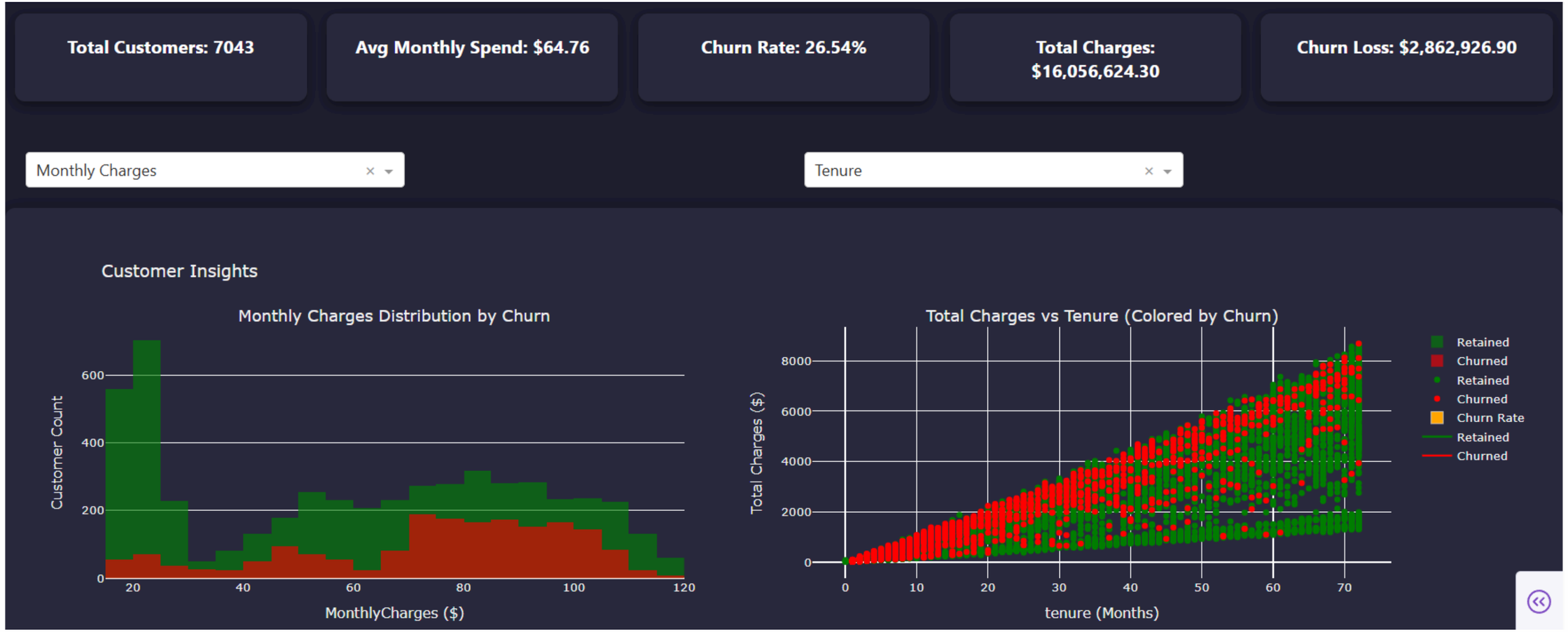

Churn, Paperless Billing, and Payment Method Analysis
PaymentMethod=2
PaymentMethod=3
PaymentMethod=0
PaymentMethod=1
Total Charges
20k
15k
10k
5k
0
50
100
Monthly Charges
Churn
0
1


Churn Rate by Internet Service Type
Churn Rate
1
0.8
0.6
0.4
0.2
0
Fiber optic
DSL
No
InternetServiceType
InternetServiceType
Fiber optic
DSL
No

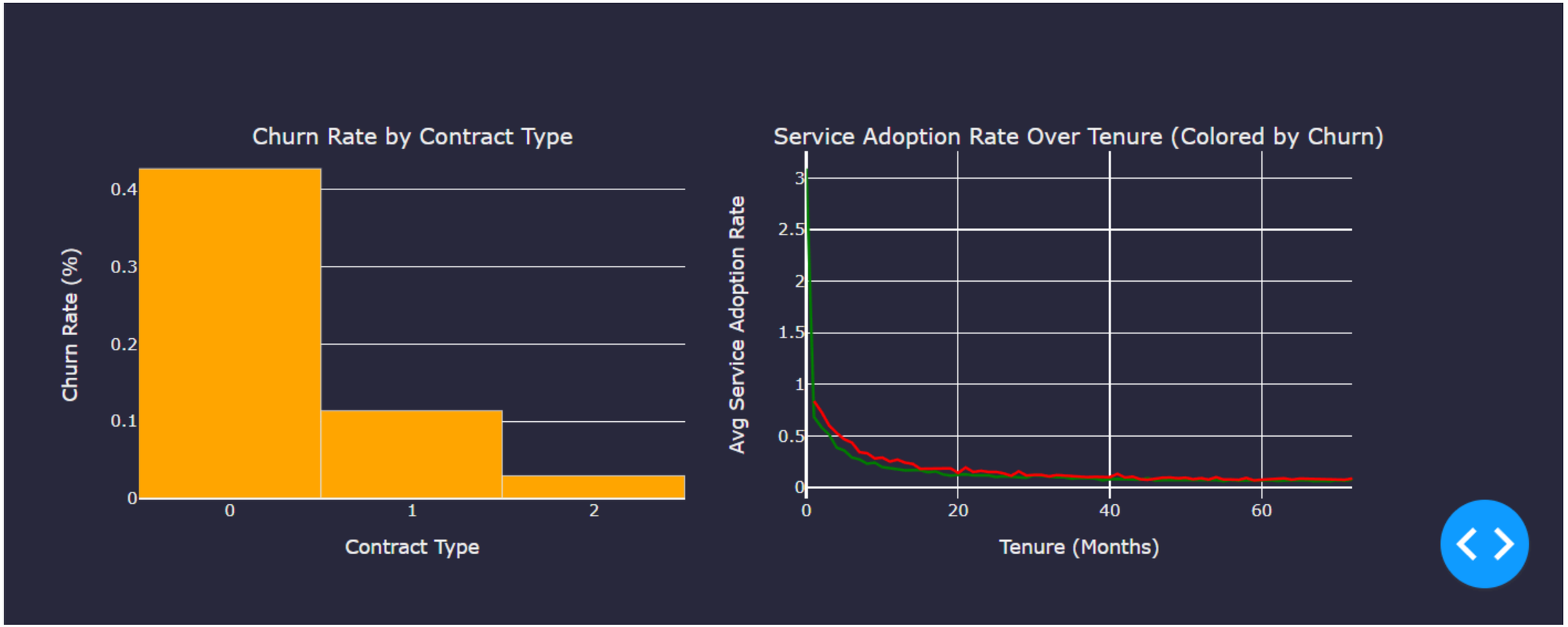

Churn Rate by Total Services and Average Monthly Spend
Avg Monthly Spend
8000
7000
6000
5000
4000
3000
2000
1000
0
0
1
2
3
4
5
6
7
8
Total Services
Churn Rate
1
0.8
0.6
0.4
0.2
0


Churn Rate by TechSupport and OnlineSecurity
Has Online Security
1
0.8
0.6
0.4
0.2
0
0
0.2
0.4
0.6
0.8
1
Has Tech Support
Churn Rate
1
0.8
0.6
0.4
0.2
0

DashBoard


Total Customers: 7043
Avg Monthly Spend: $64.76
Churn Rate: 26.54%
Total Charges: $16,056,624.30
Churn Loss: $2,862,926.90
Monthly Charges
Tenure
Customer Insights
Monthly Charges Distribution by Churn
Customer Count
MonthlyCharges ($)
Total Charges vs Tenure (Colored by Churn)
Total Charges ($)
tenure (Months)
Retained
Churned
Retained
Churned
Churn Rate
Retained
Churned

DashBoard


Churn Rate by Contract Type
Churn Rate (%)
0.4
0.3
0.2
0.1
0
0
1
2
Contract Type
Service Adoption Rate Over Tenure (Colored by Churn)
Avg Service Adoption Rate
3
2.5
2
1.5
1
0.5
0
0
20
40
60
Tenure (Months)

Machine Learning Models and Neural Network

Models

- LogisticRegression
- DecisionTree
- RandomForest
- SVM
- KNN
- GradientBoosting
- Bagging
- ExtraTrees
- AdaBoost

```python
def train_all_classifiers(x_train, x_test, y_train, y_test):
    models = {
        "LogisticRegression": LogisticRegression(max_iter=1000, C=1.0, solver='lbfgs'),
        "DecisionTree": DecisionTreeClassifier(criterion='entropy', max_depth=10, min_samples_split=10, random_state=42),
        "RandomForest": RandomForestClassifier(n_estimators=200, max_depth=10, min_samples_split=10, random_state=42),
        "SVM": SVC(C=1.0, kernel='rbf', gamma='scale', probability=True),
        "KNN": KNeighborsClassifier(n_neighbors=5, weights='uniform', metric='minkowski'),
        "GradientBoosting": GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42),
        "Bagging": BaggingClassifier(n_estimators=100, max_samples=0.8, max_features=0.8, random_state=42),
        "ExtraTrees": ExtraTreesClassifier(n_estimators=200, max_depth=10, min_samples_split=10, random_state=42),
        "AdaBoost": AdaBoostClassifier(n_estimators=200, learning_rate=0.5, random_state=42),
    }
```

```python
results, best_model_name, best_accuracy, best_model_path = train_all_classifiers(x_train, x_test, y_train, y_test)
```

```
LogisticRegression    Accuracy = 0.7999  F1 Score = 0.5877  Recall = 0.5360
DecisionTree          Accuracy = 0.7722  F1 Score = 0.5585  Recall = 0.5413
RandomForest          Accuracy = 0.8034  F1 Score = 0.5944  Recall = 0.5413
SVM                   Accuracy = 0.7807  F1 Score = 0.5040  Recall = 0.4187
KNN                   Accuracy = 0.7757  F1 Score = 0.5511  Recall = 0.5173
GradientBoosting      Accuracy = 0.7977  F1 Score = 0.5803  Recall = 0.5253
Bagging               Accuracy = 0.7942  F1 Score = 0.5579  Recall = 0.4880
ExtraTrees            Accuracy = 0.8013  F1 Score = 0.5770  Recall = 0.5093
AdaBoost              Accuracy = 0.7977  F1 Score = 0.5740  Recall = 0.5120
```

The Models Used

In this project, we implemented and tested various machine learning models including Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Gradient Boosting, Bagging, Extra Trees, and AdaBoost. All models were trained using the same dataset and evaluated using consistent metrics.

Accuracy & Metrics

This output shows the evaluation results. We used accuracy, F1 score, and recall to compare performance. As shown, the best performing model was: [RandomForest] with an accuracy of [0.8034].

```python
smote = SMOTE(random_state=514, k_neighbors=5)
X_balanced, y_balanced = smote.fit_resample(X, y)

from collections import Counter
Counter(y_balanced)
```

```
Counter({0: 5174, 1: 5174})
```

```python
x_train, x_test, y_train, y_test = train_test_split(X_balanced, y_balanced, test_size=0.2, random_state=514,shuffle=True)
x_train.shape, x_test.shape
```

```
((8278, 10), (2070, 10))
```

We applied SMOTE (Synthetic Minority Over-sampling Technique) to address class imbalance in the dataset. It generates synthetic samples for the minority class by interpolating between existing minority samples and their neighbors

After applying SMOTE, we verified the new class distribution using Counter(y_balanced), and then split the balanced dataset into training and testing sets.

```python
results, best_model_name, best_accuracy, best_model_path = train_all_classifiers(x_train, x_test, y_train, y_test)
```

```
LogisticRegression    Accuracy = 0.7657  F1 Score = 0.7745  Recall = 0.8025
DecisionTree          Accuracy = 0.7783  F1 Score = 0.7913  Recall = 0.8382
RandomForest          Accuracy = 0.7957  F1 Score = 0.8022  Recall = 0.8266
SVM                   Accuracy = 0.7686  F1 Score = 0.7812  Recall = 0.8237
KNN                   Accuracy = 0.7845  F1 Score = 0.7950  Recall = 0.8333
GradientBoosting      Accuracy = 0.7913  F1 Score = 0.7972  Recall = 0.8179
Bagging               Accuracy = 0.8135  F1 Score = 0.8157  Recall = 0.8227
ExtraTrees            Accuracy = 0.7870  F1 Score = 0.7961  Recall = 0.8295
AdaBoost              Accuracy = 0.7729  F1 Score = 0.7822  Recall = 0.8131
```

Balancing the dataset improved model fairness and performance, especially in recall and F1-score.

BorderlineSMOTE

```python
from imblearn.over_sampling import BorderlineSMOTE

smote = BorderlineSMOTE(random_state=514, k_neighbors=5)
X_balanced2, y_balanced2 = smote.fit_resample(X, y)
```

```python
x_train, x_test, y_train, y_test = train_test_split(X_balanced2, y_balanced2, test_size=0.2, random_state=514,shuffle=True)

results, best_model_name, best_accuracy, best_model_path = train_all_classifiers(x_train, x_test, y_train, y_test)
```

```
LogisticRegression    Accuracy = 0.7464  F1 Score = 0.7586  Recall = 0.7948
DecisionTree          Accuracy = 0.7618  F1 Score = 0.7887  Recall = 0.8863
RandomForest          Accuracy = 0.7884  F1 Score = 0.8064  Recall = 0.8786
SVM                   Accuracy = 0.7614  F1 Score = 0.7861  Recall = 0.8748
KNN                   Accuracy = 0.7981  F1 Score = 0.8141  Recall = 0.8815
GradientBoosting      Accuracy = 0.7812  F1 Score = 0.7953  Recall = 0.8478
Bagging               Accuracy = 0.8053  F1 Score = 0.8133  Recall = 0.8459
ExtraTrees            Accuracy = 0.7787  F1 Score = 0.7984  Recall = 0.8738
AdaBoost              Accuracy = 0.7560  F1 Score = 0.7718  Recall = 0.8227
```

BorderlineSMOTE is an enhanced version of SMOTE that focuses on samples near the decision boundary. It generates synthetic samples only for the "hard-to-classify" minority class instances, leading to better class separation and improved model performance in imbalanced scenarios.

Used when minority class samples are close to the decision boundary.

SMOTEENN

```python
from imblearn.combine import SMOTEENN

smote_enn = SMOTEENN(random_state=514)
X_balanced3, y_balanced3 = smote_enn.fit_resample(X, y)
```

```python
x_train, x_test, y_train, y_test = train_test_split(X_balanced3, y_balanced3, test_size=0.2,random_state=514,shuffle=True)

results, best_model_name, best_accuracy, best_model_path = train_all_classifiers(x_train, x_test, y_train, y_test)
```

```
LogisticRegression    Accuracy = 0.9129  F1 Score = 0.9191  Recall = 0.9225
DecisionTree          Accuracy = 0.9313  F1 Score = 0.9363  Recall = 0.9419
RandomForest          Accuracy = 0.9504  F1 Score = 0.9540  Recall = 0.9583
SVM                   Accuracy = 0.9153  F1 Score = 0.9216  Recall = 0.9285
KNN                   Accuracy = 0.9496  F1 Score = 0.9534  Recall = 0.9613
GradientBoosting      Accuracy = 0.9337  F1 Score = 0.9382  Recall = 0.9389
Bagging               Accuracy = 0.9568  F1 Score = 0.9599  Recall = 0.9627
ExtraTrees            Accuracy = 0.9440  F1 Score = 0.9483  Recall = 0.9568
AdaBoost              Accuracy = 0.9129  F1 Score = 0.9193  Recall = 0.9255
```

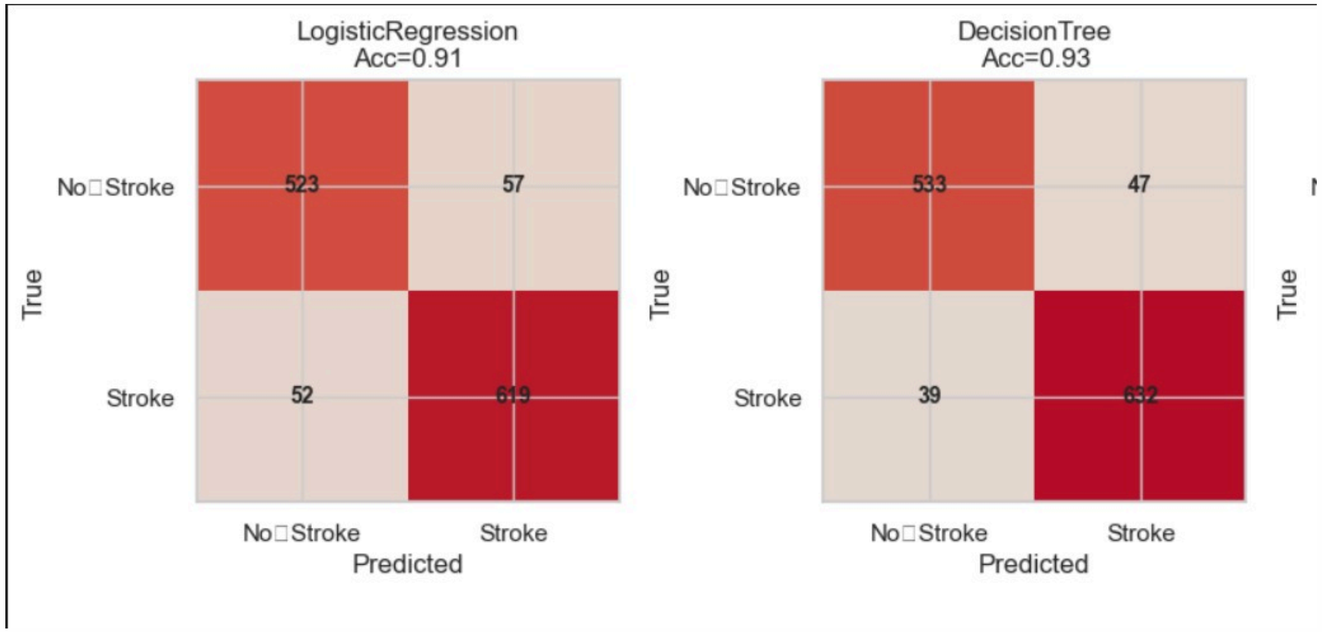
SMOTEENN combines SMOTE with Edited Nearest Neighbors (ENN). After generating synthetic samples (SMOTE), it removes noisy and ambiguous points (ENN). This hybrid method enhances data quality and helps reduce overfitting.

Used to both balance the dataset and clean noise from the data.

We experimented with different resampling techniques to improve model robustness and handle class imbalance effectively.

Confusion Matrix Visualization for All Models

```python
def plot_confusion_grid(results, y_true, labels=("No Stroke", "Stroke")):
    n_models = len(results)
    n_cols   = 3
    n_rows   = int(np.ceil(n_models / n_cols))

    fig, axes = plt.subplots(n_rows, n_cols,figsize=(4*n_cols, 4*n_rows),squeeze=False)
    axes = axes.ravel()

    vmax = max(confusion_matrix(y_true, res["pred"]).max()
               for res in results.values())
    cmap = plt.get_cmap("coolwarm")

    for ax, (name, res) in zip(axes, results.items()):
        cm = confusion_matrix(y_true, res["pred"])
        im = ax.imshow(cm, cmap=cmap, vmin=-vmax, vmax=vmax)

        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                ax.text(j, i, cm[i, j],ha="center", va="center",fontsize=10, fontweight="bold")

        ax.set_title(f"{name}\nAcc={res['acc']:.2f}")
        ax.set_xlabel("Predicted")
        ax.set_ylabel("True")
        ax.set_xticks([0, 1], labels)
        ax.set_yticks([0, 1], labels)

    for k in range(len(results), len(axes)):
        axes[k].axis("off")

    fig.colorbar(im, ax=axes[:len(results)],fraction=0.02, pad=0.02, aspect=30,label="Count")
    plt.tight_layout()
    plt.show()
```



We visualized confusion matrices for all models to compare performance in one view.
Diagonal = correct predictions
Off-diagonal = misclassifications
Higher diagonal values = better model

This helped us spot which models performed best and which had more false predictions

Each matrix shows how well a model classified the "Stroke" vs "No-Stroke" cases — darker diagonals mean better accuracy!

```python
def train_model(model, train_loader, test_loader, criterion, optimizer, epochs=50, min_delta=0.025, patience=5):
    best_loss = float('inf')
    wait = 0

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for inputs, labels in train_loader:
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        avg_loss = running_loss / len(train_loader)

        # Early stopping
        if best_loss - avg_loss > min_delta:
            best_loss = avg_loss
            wait = 0
        else:
            wait += 1
            if wait >= patience:
                print(f"Early stopping at epoch {epoch+1}")
                break
```

```python
# Model evaluation on test data
model.eval()
all_preds = []
all_labels = []
with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs)
        preds_binary = (outputs > 0.5).float()
        all_preds.extend(preds_binary.squeeze().cpu().numpy())
        all_labels.extend(labels.squeeze().cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

accuracy = (all_preds == all_labels).mean()
precision = precision_score(all_labels, all_preds, zero_division=0)
recall = recall_score(all_labels, all_preds, zero_division=0)
f1 = f1_score(all_labels, all_preds)

return accuracy, precision, recall, f1
```

Model Architecture & Code Snippet

We built a custom neural network using PyTorch. The model consists of:

Input layer based on number of features

Two hidden layers with ReLU activation

Batch Normalization and Dropout for regularization

Final sigmoid layer for binary classification

Why Deep Learning?

To capture more complex, non-linear patterns in the data that classical ML models might miss.

Tech Used:

PyTorch (nn.Module, DataLoader, BCELoss, Adam Optimizer)

Early stopping technique to prevent overfitting

Batch size = 32, learning rate = 0.001

```
    model.eval()
    with torch.no_grad():
        y_test_pred = model(X_test).round()
        f1 = f1_score(y_test, y_test_pred)
        recall = recall_score(y_test, y_test_pred)
        predicted = (y_test_pred >= 0.5).float()
        accuracy = (predicted.eq(y_test).sum() / y_test.shape[0]).item()
        print(f"F1 Score: {f1:.4f}, Recall: {recall:.4f} , Test Accuracy: {accuracy*100:.2f}%")
6]

F1 Score: 0.9240, Recall: 0.9240 , Test Accuracy: 91.85%
```

The model was trained for several epochs using binary cross-entropy loss. Below are the final evaluation metrics on the test set:

Deep learning gave us a flexible and powerful way to model complex patterns in medical data.

```python
from itertools import product

def grid_search(param_grid):
    best_acc = 0
    best_params = None
    for hidden1, hidden2, lr in product(param_grid['hidden1'], param_grid['hidden2'], param_grid['lr']):
        model = nn.Sequential(
            nn.Linear(X_train.shape[1], hidden1),
            nn.ReLU(),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1),
            nn.Sigmoid()
        )
        optimizer = optim.Adam(model.parameters(), lr=lr)
        criterion = nn.BCELoss()
        acc, precision, recall, f1  = train_model(model, train_loader, test_loader, criterion, optimizer)
        print(f"Params: h1={hidden1}, h2={hidden2}, lr={lr:.4f} => Acc={acc:.4f} , f1={f1:.4f},=> precision={precision:.4f} , => recall={recall:.4f}")
        if acc > best_acc:
            best_acc = acc
            best_params = {'hidden1': hidden1, 'hidden2': hidden2, 'lr': lr}
    print(f"Best Accuracy: {best_acc:.4f} with params {best_params}")


param_grid = {
    'hidden1': [32, 64, 128, 256],
    'hidden2': [32, 64, 128, 256],
    'lr': [0.01, 0.001, 0.0001]
}

grid_search(param_grid)
```

Grid Search Code

We used a manual grid search approach using itertools.product to find the best combination of hyperparameters for our deep learning model.

What we tuned:

hidden1: Number of neurons in the first hidden layer

hidden2: Number of neurons in the second hidden layer

lr: Learning rate for the optimizer

The model was re-trained for every combination, and the best performance was tracked based on accuracy.

```
Early stopping at epoch 7
Params: h1=32, h2=32, lr=0.0100 => Acc=0.9145 , f1=0.9192,=> precision=0.9312 , => recall=0.9076
Early stopping at epoch 7
Params: h1=32, h2=32, lr=0.0010 => Acc=0.9169 , f1=0.9247,=> precision=0.8987 , => recall=0.9523
Early stopping at epoch 19
Params: h1=32, h2=32, lr=0.0001 => Acc=0.9065 , f1=0.9143,=> precision=0.8991 , => recall=0.9300
Early stopping at epoch 7
Params: h1=32, h2=64, lr=0.0100 => Acc=0.9281 , f1=0.9345,=> precision=0.9132 , => recall=0.9568
Early stopping at epoch 7
Params: h1=32, h2=64, lr=0.0010 => Acc=0.9105 , f1=0.9154,=> precision=0.9280 , => recall=0.9031
Early stopping at epoch 12
Params: h1=32, h2=64, lr=0.0001 => Acc=0.9105 , f1=0.9179,=> precision=0.9033 , => recall=0.9329
Early stopping at epoch 7
Params: h1=32, h2=128, lr=0.0100 => Acc=0.9185 , f1=0.9261,=> precision=0.9013 , => recall=0.9523
Early stopping at epoch 7
Params: h1=32, h2=128, lr=0.0010 => Acc=0.9129 , f1=0.9174,=> precision=0.9336 , => recall=0.9016
Early stopping at epoch 13
Params: h1=32, h2=128, lr=0.0001 => Acc=0.9097 , f1=0.9170,=> precision=0.9043 , => recall=0.9300
Early stopping at epoch 7
Params: h1=32, h2=256, lr=0.0100 => Acc=0.9145 , f1=0.9183,=> precision=0.9420 , => recall=0.8957
Early stopping at epoch 7
Params: h1=32, h2=256, lr=0.0010 => Acc=0.9153 , f1=0.9201,=> precision=0.9313 , => recall=0.9091
Early stopping at epoch 15
Params: h1=32, h2=256, lr=0.0001 => Acc=0.9121 , f1=0.9188,=> precision=0.9107 , => recall=0.9270
Early stopping at epoch 12
...
Params: h1=256, h2=256, lr=0.0010 => Acc=0.9281 , f1=0.9320,=> precision=0.9449 , => recall=0.9195
Early stopping at epoch 13
Params: h1=256, h2=256, lr=0.0001 => Acc=0.9145 , f1=0.9194,=> precision=0.9299 , => recall=0.9091
Best Accuracy: 0.9392 with params {'hidden1': 256, 'hidden2': 256, 'lr': 0.01}
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Below are some of the results printed during the grid search process showing accuracy, F1, precision, and recall for each configuration.

Hyperparameter tuning is key to optimizing neural network performance and avoiding underfitting or overfitting.

Updating the Hyperparameter Grid Dynamically

```python
def update_param_grid(param_grid, updates):
    for key, new_values in updates.items():
        param_grid[key] = new_values
    return param_grid

param_grid = {
    'hidden1': [32, 64],
    'hidden2': [32, 64, 128],
    'lr': [0.001, 0.0005]
}

new_params = {
    'hidden1': [64, 128],
    'lr': [0.0001, 0.0005]
}

param_grid = update_param_grid(param_grid, new_params)
grid_search(param_grid)
```

Updating the Parameter Grid

We created a reusable function update_param_grid() to dynamically update our hyperparameter combinations. This allowed us to refine our search space based on initial results.

What this does:

Takes the current param_grid and applies changes from a new_params dictionary.

Saves time by avoiding full redefinition and allows easier experimentation.

```
Early stopping at epoch 14
Params: h1=64, h2=32, lr=0.0001 => Acc=0.9113 , f1=0.9184,=> precision=0.9058 , => recall=0.9314
Early stopping at epoch 8
Params: h1=64, h2=32, lr=0.0005 => Acc=0.9105 , f1=0.9154,=> precision=0.9280 , => recall=0.9031
Early stopping at epoch 12
Params: h1=64, h2=64, lr=0.0001 => Acc=0.9089 , f1=0.9163,=> precision=0.9030 , => recall=0.9300
Early stopping at epoch 11
Params: h1=64, h2=64, lr=0.0005 => Acc=0.9105 , f1=0.9155,=> precision=0.9267 , => recall=0.9046
Early stopping at epoch 12
Params: h1=64, h2=128, lr=0.0001 => Acc=0.9145 , f1=0.9204,=> precision=0.9184 , => recall=0.9225
Early stopping at epoch 10
Params: h1=64, h2=128, lr=0.0005 => Acc=0.9169 , f1=0.9224,=> precision=0.9238 , => recall=0.9210
Early stopping at epoch 14
Params: h1=128, h2=32, lr=0.0001 => Acc=0.9081 , f1=0.9146,=> precision=0.9112 , => recall=0.9180
Early stopping at epoch 12
Params: h1=128, h2=32, lr=0.0005 => Acc=0.9177 , f1=0.9227,=> precision=0.9290 , => recall=0.9165
Early stopping at epoch 11
Params: h1=128, h2=64, lr=0.0001 => Acc=0.9097 , f1=0.9170,=> precision=0.9043 , => recall=0.9300
Early stopping at epoch 12
Params: h1=128, h2=64, lr=0.0005 => Acc=0.9161 , f1=0.9225,=> precision=0.9137 , => recall=0.9314
Early stopping at epoch 10
Params: h1=128, h2=128, lr=0.0001 => Acc=0.9121 , f1=0.9183,=> precision=0.9156 , => recall=0.9210
Early stopping at epoch 7
Params: h1=128, h2=128, lr=0.0005 => Acc=0.9113 , f1=0.9166,=> precision=0.9242 , => recall=0.9091
Best Accuracy: 0.9177 with params {'hidden1': 128, 'hidden2': 32, 'lr': 0.0005}
```

New Grid Search Results

These updated values led to better-performing models. The results printed below show how the new hyperparameters improved performance in some configurations.

Why it matters:

This iterative tuning approach helped us find an optimal balance between model complexity and generalization.

```python
def random_search(param_space, n_iter=10):
    best_acc = 0
    best_params = None
    for _ in range(n_iter):
        hidden1 = random.choice(param_space['hidden1'])
        hidden2 = random.choice(param_space['hidden2'])
        lr = random.choice(param_space['lr'])

        model = nn.Sequential(
            nn.Linear(X_train.shape[1], hidden1),
            nn.ReLU(),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, 1),
            nn.Sigmoid()
        )
        criterion = nn.BCELoss()
        optimizer = optim.Adam(model.parameters(), lr=lr)
        acc, precision, recall, f1  = train_model(model, train_loader, test_loader, criterion, optimizer)
        print(f"Params: h1={hidden1}, h2={hidden2}, lr={lr:.4f} => Acc={acc:.4f} , f1={f1:.4f},=> precision={precision:.4f} , => recall={recall:.4f}")
        if acc > best_acc:
            best_acc = acc
            best_params = {'hidden1': hidden1, 'hidden2': hidden2, 'lr': lr}
    print(f"Best Accuracy: {best_acc:.4f} with params {best_params}")


param_space = {
    'hidden1': [32, 64, 128, 256],
    'hidden2': [32, 64, 128, 256],
    'lr': [0.01, 0.001, 0.0001]
}
random_search(param_space, n_iter=10)
```

Random Search Code

We implemented a random search strategy to explore a wide hyperparameter space with less computational cost compared to full grid search.

Why Random Search?

Faster than grid search when the parameter space is large.

Often finds near-optimal solutions with fewer trials.

Adds randomness that can lead to discovering unexpected good combinations.

How it works:

Randomly selects values for hidden1, hidden2, and lr for a fixed number of iterations (n_iter=10).

Trains the model on each combination and tracks performance.

Below are the evaluation results printed for each random combination. The best accuracy achieved was:

```
Early stopping at epoch 12
Params: h1=256, h2=32, lr=0.0001 => Acc=0.9105 , f1=0.9168,=> precision=0.9141 , => recall=0.9195
Early stopping at epoch 12
Params: h1=128, h2=256, lr=0.0100 => Acc=0.9257 , f1=0.9298,=> precision=0.9419 , => recall=0.9180
Early stopping at epoch 7
Params: h1=32, h2=32, lr=0.0100 => Acc=0.9201 , f1=0.9264,=> precision=0.9156 , => recall=0.9374
Early stopping at epoch 7
Params: h1=32, h2=32, lr=0.0010 => Acc=0.9129 , f1=0.9188,=> precision=0.9182 , => recall=0.9195
Early stopping at epoch 11
Params: h1=128, h2=256, lr=0.0001 => Acc=0.9137 , f1=0.9198,=> precision=0.9170 , => recall=0.9225
Early stopping at epoch 12
Params: h1=256, h2=64, lr=0.0100 => Acc=0.9281 , f1=0.9328,=> precision=0.9342 , => recall=0.9314
Early stopping at epoch 12
Params: h1=256, h2=256, lr=0.0100 => Acc=0.9225 , f1=0.9260,=> precision=0.9484 , => recall=0.9046
Early stopping at epoch 13
Params: h1=64, h2=64, lr=0.0001 => Acc=0.9137 , f1=0.9202,=> precision=0.9122 , => recall=0.9285
Early stopping at epoch 8
Params: h1=128, h2=256, lr=0.0100 => Acc=0.9217 , f1=0.9273,=> precision=0.9232 , => recall=0.9314
Early stopping at epoch 11
Params: h1=64, h2=128, lr=0.0001 => Acc=0.9097 , f1=0.9178,=> precision=0.8963 , => recall=0.9404
Best Accuracy: 0.9281 with params {'hidden1': 256, 'hidden2': 64, 'lr': 0.01}
```